

Portfolio Specification

Iris Flower Data Set



NOVEMBER 28 2025

C00286249

Hafza Abdullahi

GitHub Repository: <https://github.com/Hafza-Abdullahi/DataScienceYear4>

Stream lit App: <https://hafza-abdullahi-datascienceyear4-codeirisflowermlapp-tmfhe8.streamlit.app/>

Table of Contents

Contents

Table of Contents.....	2
What is the Iris Flower classification	3
Dataset Structure.....	3
Statistical summary.....	4
Sepal Length Difference	5
Sepal Width Differences.....	6
Petal Length Differences	7
Petal width Differences	7
Summary of feature differences	8
Overview of the data using seaborn.....	9
Training the model.....	10
Preparing data for training using NumPy.....	10
Goal	11
Simple Classification rules using petal length	12
Modelling using logistic regression	14
Project Report.....	16
Technology Stack and Reasoning.....	16
Advanced Model Selection & Comparison.....	16
Final Model Validation & Error Analysis	17
Deployment	18
Conclusion	19
References	19
Image References	20

What is the Iris Flower classification

The Iris flower data set or Fisher's Iris data set is a data set used and made famous by the British statistician and biologist Ronald Fisher in his 1936 paper.



Figure 1: Iris flower

Dataset Structure

The dataset contains 3 species of Iris Flowers: setosa, versicolor, virginica. Each of these species has 50 samples which differ in these 4 categories

- Sepal length
- Sepal width
- Petal length
- petal length

	Sepal length	Sepal width	Petal length	Petal width	Class
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
⋮	⋮	⋮	⋮	⋮	⋮
150	5.9	3.0	5.1	1.8	virginica

Figure 2: table containing the categories

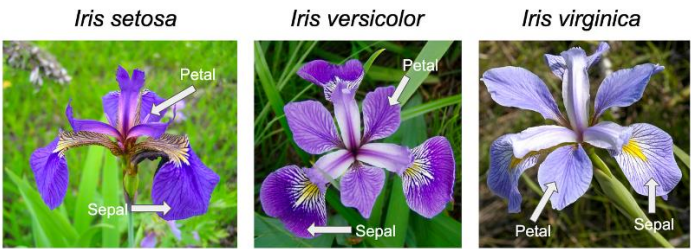


Figure 3: labelling of the flower parts

Statistical summary

```
dataFrame.describe()
```

[37] ✓ 0.0s

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

Figure 4: displays standard deviations of each measurement

Somethings to note from this complete data

- 25% of all petals are shorter than 1.6 cm and smaller than 0.3 cm
- Smallest petal is 1.0 cm long × 0.1 cm wide
- Smallest sepal is 4.3 cm long × 2.0 cm wide
- Median Petal length (4.35 cm) is much higher than mean (3.76 cm)
- The Largest petal is 6.9 cm long × 2.5 cm wide
- The Largest sepal is 7.9 cm long × 4.4 cm wide
- Petal measurements show much wider ranges than sepal so they are going to be the most important
- Potential Clustering: The difference between the mean and median for petal length shows we might have unique groups

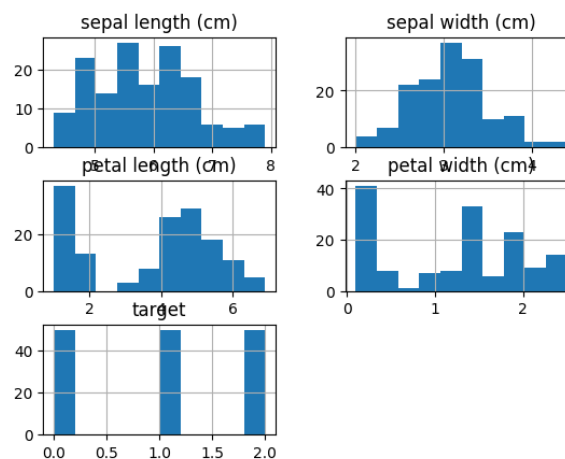


Figure 5: Histograms of features

Task

Correctly identify an iris flower to the correct iris species using the sepal width and length and the petal length and width. This is a multiclass classification problem

How? Lets compare Features of the target to each category in the dataset using seaborn, a library built for better visualization of data. It has better plotting tools and hue changes compared to matplotlib library. It's also cleaner and easier to use since it handles automatic styling and spacing.

Sepal Length Difference

The legend has the names of the species names instead of the numbers, but what does this graph mean? We can clearly see that setosa flowers have the shortest sepal length while versicolor has medium and virginica being the group with the largest. This is clear separation and difference we can use these to identify what species a flower belongs to.

Note: there is an overlap between versicolor and virginica so using sepal length alone to identify is not enough.

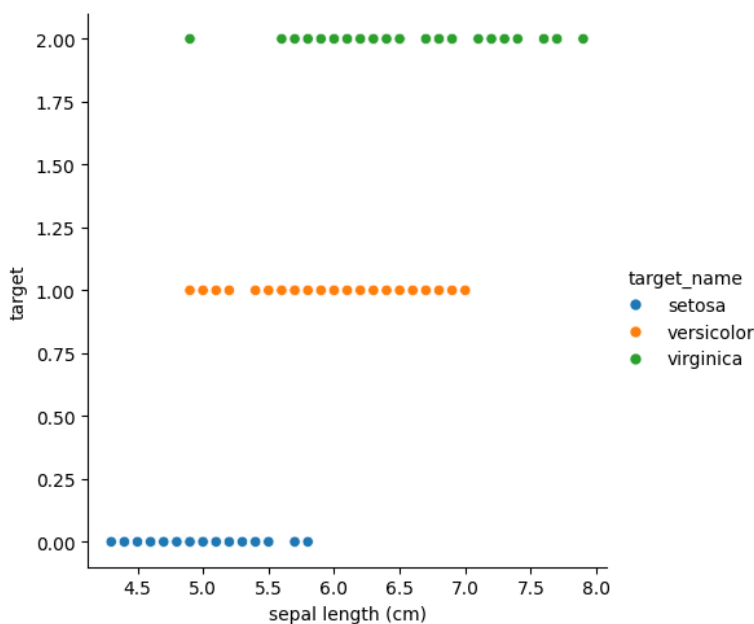


Figure 6: showing clear distinctions between the species when it comes to sepal length

Sepal Width Differences

A lot more overlapping can be seen with petal width so this is not a good unique identifier but still we can see versicolor has the smallest sepal width and setosa has the biggest

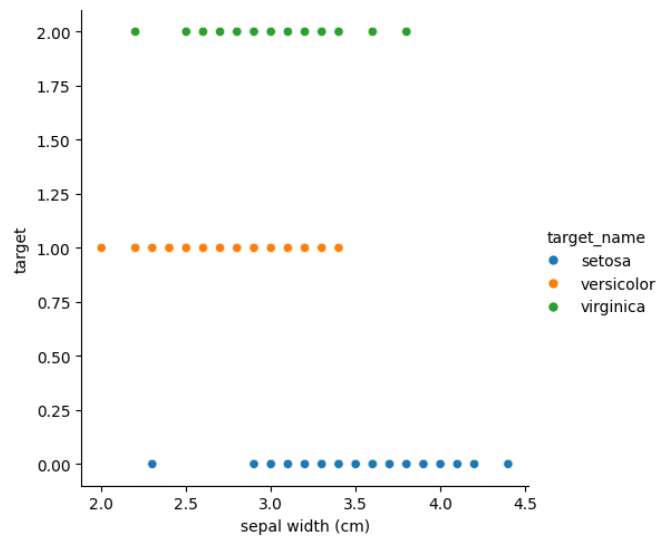


Figure 6: showing clear distinctions between the species when it comes to sepal width

Petal Length Differences

This is a great unique identifier for setosa, since it doesn't overlap at all with the two other species. This means we can tell setosa flowers just based off the petal length!

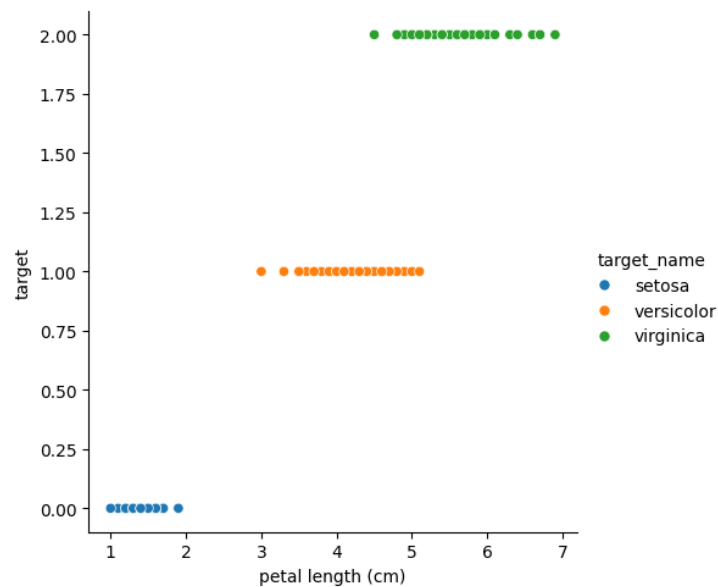


Figure 6: showing clear distinctions for setosa when it comes to petal length

Petal width Differences

Like petal length, petal width is also a unique identifier for setosa flowers

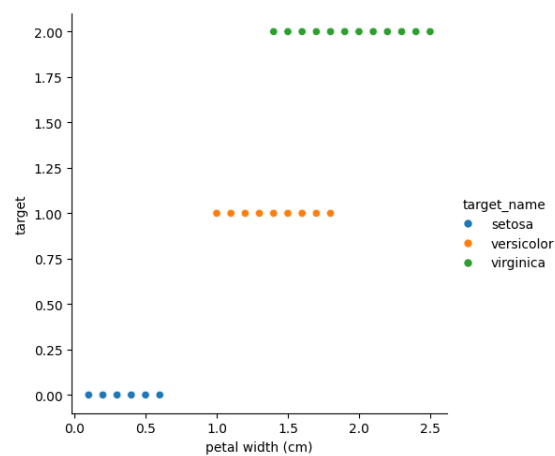


Figure 7: showing clear distinctions for setosa when it comes to petal width

Summary of feature differences

Unique Identifiers for Setosa:

- **Petal length** and **petal width** are the strongest unique identifiers for Setosa flowers
- Setosa has significantly smaller petals that don't overlap at all with the other two species
- This means we can instantly identify Setosa flowers just by looking at petal measurements alone, which is good! Easy identification

Challenges with Versicolor and Virginica:

- These two species overlap in all four features, making them harder to distinguish
- **Sepal length** shows some a little separation but still has noticeable overlap
- **Sepal width** has the most overlap and is the weakest identifier
- We cannot rely on any single feature to perfectly separate Versicolor from Virginica

Solution for Classification:

To identify all three species with accuracy, we need to use **all four features together** in a machine learning model. While Setosa can be easily spotted by its small petals, distinguishing between Versicolor and Virginica requires analysing the combination of sepal and petal measurements to find the subtle patterns that separate them.

Overview of the data using seaborn

code: `seaborn.pairplot(dataFrame, hue="target_name")`

In here we can see all the data features plotting against each other using seaborn pairplot, setosa flowers data can be seen in their own clusters when it comes to petal length and width, the rest of data shows their relationships.

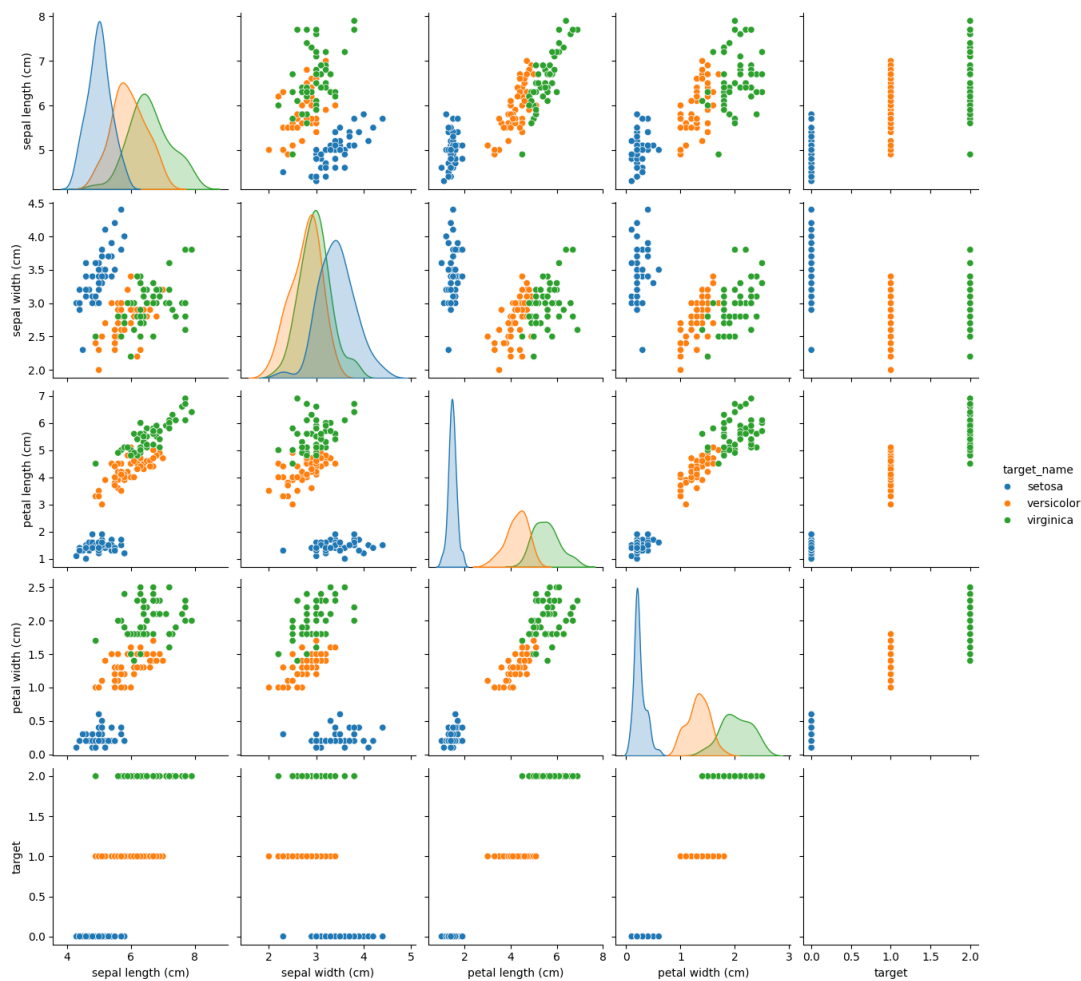


Figure 8: overview of the data plotted against each other

Training the model

Test data aka unseen data:

Before training, it's important that we keep a portion of the data that the model hasn't seen before called the **test set** to use for the final test of our model. This is to make sure we don't do a final test with data the model has seen already and knows the answer to already. By doing this we get a true measure of how well the model performs and how well it correctly identifies a brand new iris flower.

```
from sklearn.model_selection import train_test_split
#split data into 0.25 for testing and .75 for training the model
dataFrame_train, dataFrame_test = train_test_split(dataFrame, test_size=0.25)
```

Figure 9: code splitting data into train and test data

```
#we have 38 rows for testing
dataFrame_test.shape

[70] ✓ 0.0s
... (38, 7)

#we have 112 rows for training
dataFrame_train.shape

[71] ✓ 0.0s
... (112, 7)
```

Figure 10: code showing data split into 38 test rows and 112 train rows

Preparing data for training using NumPy

To clean the data we need to remove the species and target name columns and just have a NumPy array of the numbers

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	species	target_name
78	6.0	2.9	4.5	1.5	1	Iris-versicolor	versicolor
103	6.3	2.9	5.6	1.8	2	Iris-virginica	virginica
142	5.8	2.7	5.1	1.9	2	Iris-virginica	virginica
68	6.2	2.2	4.5	1.5	1	Iris-versicolor	versicolor
121	5.6	2.8	4.9	2.0	2	Iris-virginica	virginica

Figure 11: code showing before cleaning
Source: my code

Data after removing those columns

```
#clean data by removing the cols we dont want to train our model with
x_train = dataframe_train.drop(columns=["target", "target_name", "species"]).values

#display cleaned data
x_train
```

```
[76] ✓ 0.0s
```

```
... array([[6. , 2.9, 4.5, 1.5],
          [6.3, 2.9, 5.6, 1.8],
          [5.8, 2.7, 5.1, 1.9],
          [6.2, 2.2, 4.5, 1.5],
          [5.6, 2.8, 4.9, 2. ],
          [4.8, 3.1, 1.6, 0.2],
          [5.5, 2.4, 3.8, 1.1],
          [5. , 3.5, 1.3, 0.3],
          [5.8, 2.8, 5.1, 2.4],
          [6. , 3.4, 4.5, 1.6],
          [6.9, 3.1, 5.1, 2.3],
          [5.5, 2.6, 4.4, 1.2],
          [4.7, 3.2, 1.6, 0.2],
```

Figure 12: code data ready for training
Source: my code

Goal

If we had a wheel randomly spinning what a new flower might be without seeing it, what will we get? Mathematically speaking it should guess correctly 1/3 of the time,

since there are 3 options and the classes are evenly balanced. So, our goal is to create a model that guesses the answer correctly more than 1/3 of the time

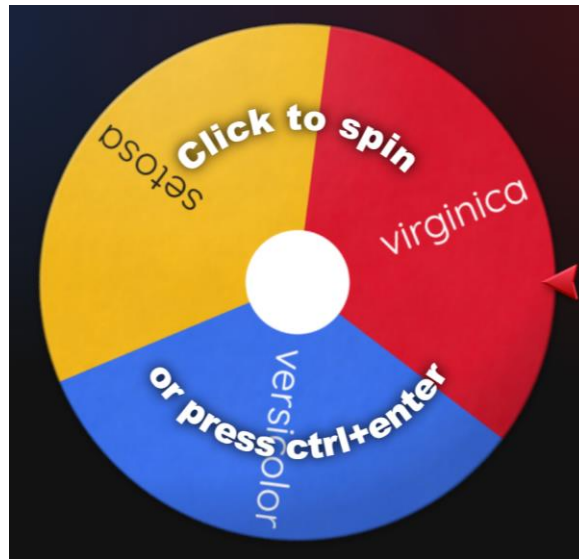


Figure 13: spinner wheel with 3 options

Simple Classification rules using petal length

Using this graph again

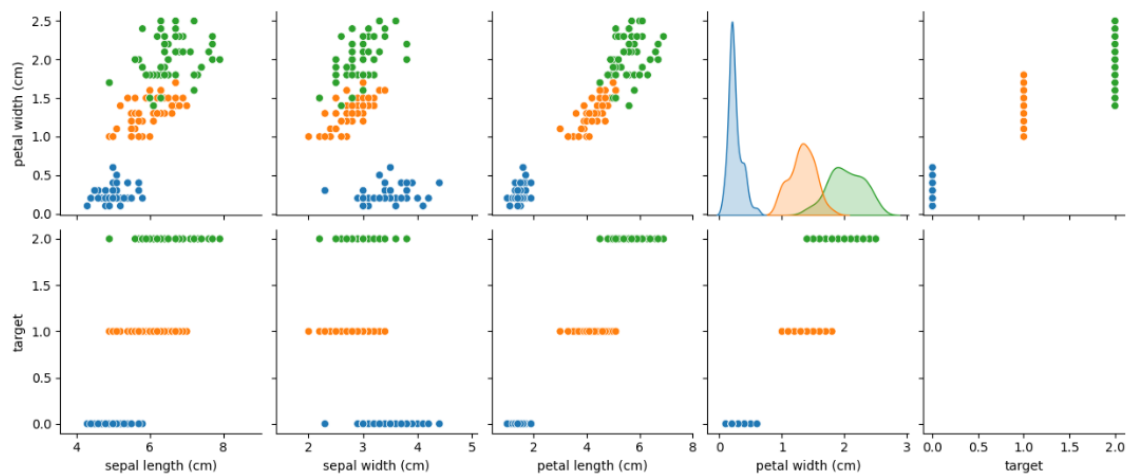


Figure 14: data features graphed against each other showing some classification

Source: my code

We can see that

- If the petal length is less than 2.5 it's a setosa flower
- If the petal length is less than 4.8 but bigger than 2.5 it's versicolor
- If both statements are false, then it's a virginica

Sample input and output without machine learning

```
def single_feature_manual_guess(petal_length):
    #Predicts based on classification rules
    if petal_length < 2.5:
        return 0
    elif petal_length < 4.8:
        return 1
    else:
        return 2

#Testing the function using values for petal length
print(single_feature_manual_guess(1.5)) #should Returns 'setosa'
print(single_feature_manual_guess(3.0)) #should Returns 'versicolor'
print(single_feature_manual_guess(5.5)) #should Returns 'virginica'
```

[136] ✓ 0.0s

```
... 0
    1
    2
```

Figure 15: single feature guessing without any learning
Source: my code

[illegible]

Figure 16: output of manual guessing with the training data
Source: my code

We can see that the manual guessing was pretty accurate

```

> nmp.mean(manual_guess == y_train)
[161] ✓ 0.0s
... np.float64(0.9464285714285714)

```

Figure 17: output of manual guessing mean

Modelling using logistic regression

Logistical regression is a classification algorithm that tries to find patterns in data and separate them by classes by drawing boundaries between them. This is perfect for our iris flower dataset which has 3 species

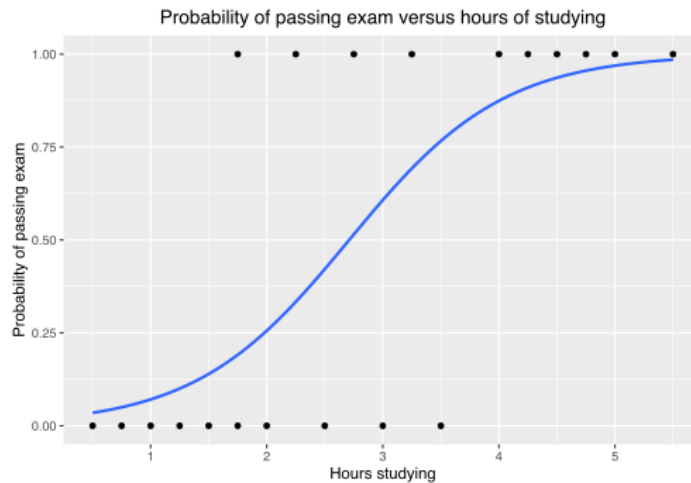


Figure 18: logistic regression

```
model = LogisticRegression()
163] ✓ 0.0s

> >
#this learns the relationship between the flower measurements and their labels
model.fit(x_train, y_train)
172] ✓ 0.0s

...
LogisticRegression ⓘ ⓘ
Parameters
penalty 'l2'
dual False
tol 0.0001
C 1.0
fit_intercept True
intercept_scaling 1
class_weight None
random_state None
solver 'lbfgs'
max_iter 100
multi_class 'deprecated'
verbose 0
warm_start False
n_jobs None
l1_ratio None

#test how well the model performs on the training data
model.score(x_train, y_train)
169] ✓ 0.0s

... 0.9732142857142857
```

Figure 19: data training using logistic regression

Source: my code

Note:

```
#xtrain, xvalidate, ytrain, yvalidate
xT, xV, yT, yV = train_test_split(x_train, y_train, test_size=0.25)

[177] ✓ 0.0s

xT.shape
[178] ✓ 0.0s
... (84, 4)

#this learns the relationship between the flower measurements and their labels using the sectioned of data to train
model.fit(xT, yT)
[179] ✓ 0.0s
...
LogisticRegression
Parameters

#test how well the model performs on the training data , which is not good
model.score(x_train, y_train)
[180] ✓ 0.0s
... 0.9553571428571429
```

Figure 20: decrease in accuracy since we used less data

Let's go back to the goal
The manual testing accuracy

```
np.mean(manual_guess == y_train)
[188] ✓ 0.0s
... np.float64(0.9464285714285714)
```

vs our model which performs better overall
note how the unseen data is less accurate

```
Unseen data

#accuracy
np.mean(y_prediction == yV)
[190]
... np.float64(0.9285714285714286)

Seen Data

#test how well the model performs on the training data
model.score(x_train, y_train)
[191]
... 0.9553571428571429
```

Project Report

Technology Stack and Reasoning

Technology Stack & Reasoning

To ensure this project meets standards, the following technologies were selected

- **Python:** Chosen as the core programming language due to its extensive ecosystem for data science. Key libraries used include **Pandas** for data manipulation, **Scikit-Learn** for modelling, and **Seaborn** for high-quality visualization.
- **VS Code (Visual Studio Code):** Selected for seamless switching between Jupyter Notebooks and Python scripts (app.py).
- **Streamlit:** Used for the deployment phase as its focus on interactivity and easy setup using github
- **GitHub:** Utilized for version control.

Advanced Model Selection & Comparison

Three distinct algorithms to test

- **Logistic Regression (LR):** A linear model, excellent as a baseline for classification tasks.
- **K-Nearest Neighbors (KNN):** A non-linear, distance-based algorithm that classifies data points based on their proximity to others.
- **Decision Tree (CART):** A rule-based algorithm that splits data into branches, mimicking human decision-making.

Methodology

Stratified K-Fold Cross-Validation

Instead of a simple train/test split which can be very biased by chance, I used 10-Fold Stratified Cross-Validation. This technique splits the training data into 10 parts, training on 9 and testing on 1, repeating this process 10 times. "Stratified" ensures every fold has the same proportion of flower species as the original dataset.

Results:

The results of the cross-validation are as follow

- **Logistic Regression:** 96.66% Accuracy (Standard Deviation: 0.04)
- **K-Nearest Neighbors:** 95.83% Accuracy (Standard Deviation: 0.05)
- **Decision Tree:** 94.16% Accuracy (Standard Deviation: 0.07)

Logistic Regression proved to be the most stable and accurate model for this dataset, due to the clear linear separation of the Setosa class and the relatively linear boundary between Versicolor and Virginica flowers.

Final Model Validation & Error Analysis

The final Logistic Regression model was trained on the full training set (80% of data) and evaluated it against the held-out Test Set (20% of data)

Confusion Matrix Analysis

To understand *where* the model makes mistakes, we visualized the results using a **Confusion Matrix Heatmap**

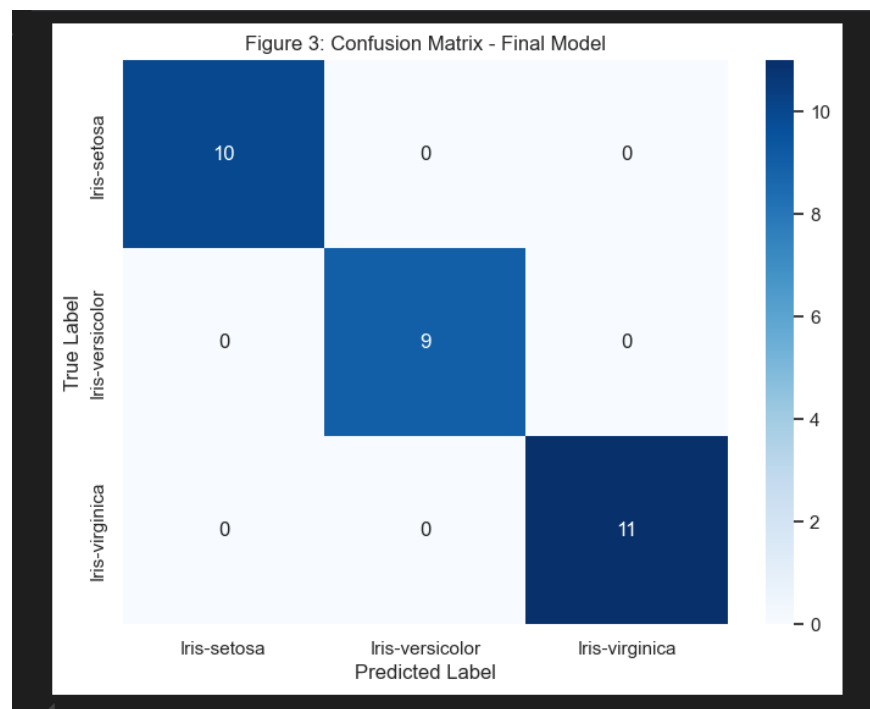


Figure 21: *Confusion Matrix Heatmap*

Analysis:

- **Setosa:** 100% correct classification. As predicted in the analysis, the distinct petal measurements make this class easy to identify and differentiate from the others
- **Versicolor & Virginica:** The matrix shows minimal misclassification between these two classes (e.g., 1 Versicolor predicted as Virginica). This aligns with the feature overlap we observed in the pair plots earlier.

Deployment

I developed a frontend application using **Streamlit**.

This app enables a user to use sliders to input flower measurements. The model runs in the background and outputs a prediction in real-time, along with a probability bar chart showing the model's confidence.

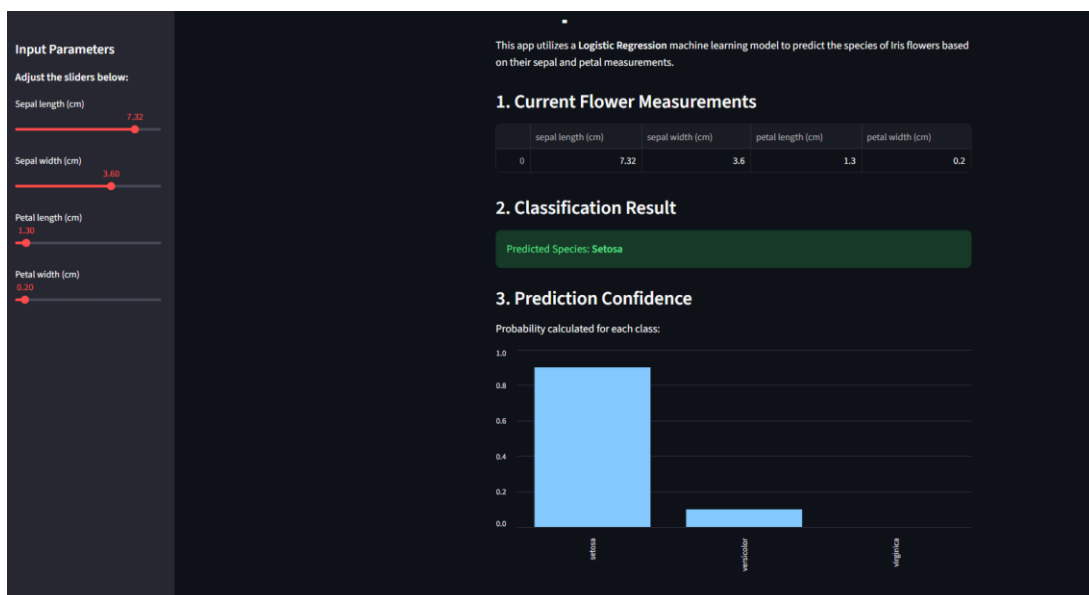


Figure 22: screenshot of stream lit app running predictions

Conclusion

This Iris Flower Classification project demonstrated how data analysis and machine learning can be used to accurately identify flower species based on measurable traits like petal length, width etc. After looking at the data, it was clear that petal length was a good unique trait for identifying Setosa, which showed no overlap with the other two classes.

This result showed that while it performed worse than the manual guessing with the unseen data, with better training it could do better. The seen data set performed better overall

References

- Iris: https://en.wikipedia.org/wiki/Iris_flower_data_set
- Iris dataset: <https://archive.ics.uci.edu/dataset/53/iris>
- scikit Documentation: https://scikit-learn.org/1.4/auto_examples/datasets/plot_ir
- Streamlit Documentation: <https://docs.streamlit.io>
- Pandas Documentation: <https://pandas.pydata.org>

Image References

- [1] <https://thursd.com/articles/iris-flower-meaning>
- [13] <https://wheelofnames.com/>
- [18] https://en.wikipedia.org/wiki/Logistic_regression